

Dossier de preuve : performance et services back-end

ShopFlow, PostgreSQL 18 : besoin, stockage, API, résultats avant/après, limites. Chaque chiffre vient d'un fichier de mesures du projet.

Pièce du dossier (slide 30)	Ce qu'elle démontre	Section
Diagnostic initial	problème observable	1
DDL et déploiement	stockage implémenté	2
SQL et trace API	accès correct aux données	3
Benchmark avant/après	effet mesuré sur le service	4
Tests de correction	invariants et fraîcheur	5
Décision finale	choix et limites justifiés (grille slide 31)	6

Besoin et périmètre	Valeur
Accès étudiés	historique d'un client (GET /commandes?limit=20) · file des commandes en attente · fiche produit · indicateurs mensuels et export · évolution du schéma (newsletter_ok)
Charge	1, 5, 10 et 20 clients (10 s, 3 s d'échauffement, 3 essais alternés) sur l'API ; mesures SQL par pgbench
Jeu de données	1 000 clients, 200 produits, 100 000 commandes, 300 000 lignes (tables de travail jusqu'à 3 000 000 de lignes)
Environnement	PostgreSQL 18.6 (Docker), API Node v22.14.0 + Redis 8.10.2, pgbench 18.6, PgBouncer 1.18 ; 8 cœurs, 7,6 Go, WSL2, tout sur la même machine
Protocole	résultat identique vérifié avant toute durée (empreinte md5 ou EXCEPT) · 3 échauffements puis 5 mesures (médiane) · tours alternés sur tables de travail, sans parallélisme · valeurs stables privilégiées (pages lues, WAL, plans) · un seul changement par série · laboratoire comparé avant/après (6 index, 0 statistique)

Synthèse : ancienne version → nouvelle version, résultat identique

#	Optimisation	Ancienne → nouvelle	Gain	Coût ou limite
1	Agrégation mensuelle (statistique)	84,6 → 45,5 ms	×1,9	statistique à maintenir (non mesuré)
2	Clients avec commande payée (EXISTS)	40,2 → 11,3 ms	×3,6	aucun coût mesuré
3	Historique client (index composé)	0,690 → 0,274 ms	×2,5	WAL d'insertion +35 %
3 bis	Idem, client très actif	23,3 → 0,256 ms	×91,1	idem
4	File en attente (index partiel)	10,1 → 0,070 ms	×145	WAL +3,2 %, index 328 kB
5	Filtre JSON (colonne typée)	30,9 → 1,0 ms	×30,4	WAL +38,9 %
5 bis	Filtre JSON (index d'expression)	30,9 → 1,1 ms	×28,0	WAL +34,6 %
5 ter	Filtre JSON @> (GIN, valeur rare)	43,9 → 2,6 ms	×17,2	WAL +120,7 %
6	Chevauchement de périodes (GiST)	27,5 → 0,168 ms	×164	WAL +54,1 %, insertions ×7,8
7	Migration NOT NULL (verrou exclusif)	3 324 → 46,3 ms	×71,8	remplissage ×2,7 plus long
8	Page avec ses lignes (N+1 → groupé)	55,3 → 43,9 ms	×1,3	gain modeste en local (21 → 2 requêtes)
9	Page profonde (OFFSET → curseur)	99,7 → 0,168 ms	×594	aucun gain à petite profondeur
10	Fiche produit (SELECT → cache Redis)	2,7 → 0,988 ms	×2,7	copie périmée jusqu'à 60 s

Ligne 7 : temps où la table est inaccessible (3 000 000 lignes) ; 9 : décalage 100 000 sur 1 000 000 de commandes ; 10 : connexion persistante. Résultat identique prouvé pour chaque ligne (section 5). Médianes ; durées variables d'une exécution à l'autre.

1. Diagnostic initial : problèmes observables

Observation	Mesure	Cause identifiée
Historique du client 42	0,229 ms	déjà rapide ; index (client_id, ...) préexistant ; le tri porte sur 100 lignes
Indicateurs mensuels	84,1 ms ; Sort externe 3 040 kB	work_mem (4 MB) trop petit et estimation fausse : 80 048 groupes prévus, 30 réels
Page de commandes avec lignes	21 requêtes SQL pour 20 commandes	N+1 : une requête de lignes par commande
Pagination profonde	OFFSET 999 000 : 944,3 ms, 1 003 945 pages	OFFSET relit ce qu'il écarte
Évolution de schéma	SET NOT NULL direct : table inaccessible 3 324 ms (3 000 000 lignes)	relecture complète sous verrou exclusif
Rapports sur l'instance OLTP	p95 des commandes +64 %	calcul analytique concurrent sur la même instance

Observation (Jour 5). pg_stat_statements classe le rapport mensuel en tête (10 appels, 1 117 ms cumulés) ; le N+1 y pèse 369 ms (20 000 appels de 0,018 ms) et **n'apparaît pas** dans le journal des requêtes lentes (0 ligne), alors que l'API mesure 14,2 ms de SQL par page : son coût est dans les allers-retours. pg_stat_activity (slide 8) : une session « idle in transaction » (ClientRead) bloque l'ALTER d'une autre (Lock / relation, pg_blocking_pids = la première). Laboratoire : autovacuum on, journal lent -1 (désactivé).

2. DDL et déploiement : stockage implémenté

Objet	DDL (extrait)	Déploiement et retour arrière	Preuve
Schéma ShopFlow	01_schema.sql, 02_donnees.sql	base de laboratoire PostgreSQL 18.6	1 000 / 200 / 100 000 / 300 000 lignes
Index historique (001)	CREATE INDEX ... (client_id, created_at DESC, id DESC) INCLUDE (statut, total)	version laboratoire, version production CONCURRENTLY hors transaction, contrôle indisvalid, DOWN	testée sur base jetable ; non appliquée au laboratoire
Index file d'attente (002)	CREATE INDEX ... (created_at, id) WHERE statut = 'en_attente'	idem ; DROP INDEX au retour	testée sur base jetable ; non appliquée
Colonne newsletter_ok (4 bis)	ADD COLUMN nullable → DEFAULT false → lots de 200 → CHECK ... NOT VALID → VALIDATE → SET NOT NULL	lock_timeout 2 s ; retour : retrait des contraintes puis DROP COLUMN (sur la copie)	1000 clients, 0 NULL, 0 écart (md5 aec47d5bdba0) ; verrou : 55P03 après 2,0 s

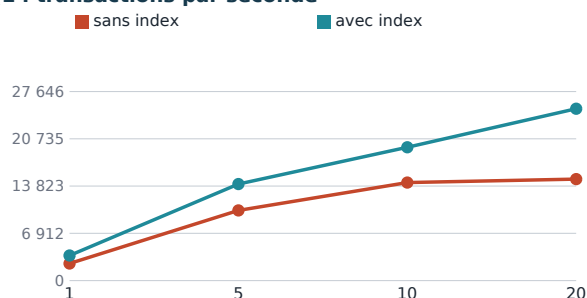
Migration à 3 000 000 de lignes (même état final, md5 b227f2377b62) : temps sous verrou exclusif 3 324 → 46,3 ms ; écriture la plus longue bloquée 30 989 → 416 ms ; remplissage 57 → 156 s (coût de la méthode). Pendant VALIDATE, lecture, UPDATE et INSERT concurrents passent ; pendant SET NOT NULL, une lecture est bloquée (55P03).

3. SQL et trace API : accès correct aux données

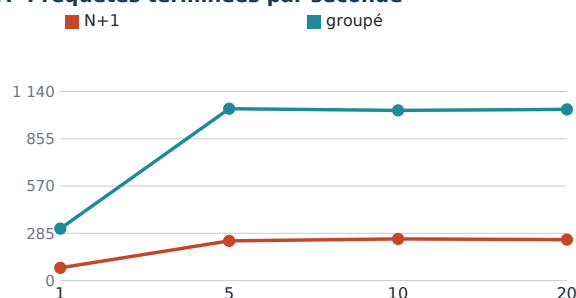
Élément	Contenu	Preuve
Contrat de l'endpoint	GET /commandes ; réponse data / hasNextPage / nextCursor ; ordre created_at DESC puis id DESC ; limite 20 (défaut), 100 (max) ; ids et montants en chaînes ; client issu du contexte autorisé	vérifié ; limit 101 et 0 : 400
SQL curseur	WHERE client_id=\$1 AND (created_at,id)<(\$2,\$3) ORDER BY commandes.created_at DESC, commandes.id DESC LIMIT 21 (la 21e ligne donne hasNextPage)	page 2 identique à OFFSET ; parcours 5 pages, 100 ids uniques
SQL chargement groupé	SELECT ... FROM lignes WHERE commande_id = ANY(\$1::bigint[])	réponse identique au N+1 (5, 20, 50, 100 commandes)
Trace (Traceld → SQL)	21 requêtes (N+1), sqlMs 40,9 → 2 requêtes (groupé), sqlMs 3,5	Traceld 5aa4798c... / 41f80089...
Dates identiques	page 1 : 2042, 1042 ; page 2 : 42, 86042 ; avec la seule date la commande 42 est perdue	paire (created_at, id) nécessaire
Droits	401 sans jeton ; 400 avec client_id dans l'URL ; 400 avec un curseur falsifié ; un curseur signé n'est pas une autorisation	401 / 400 / 400

4. Benchmark avant/après : effet mesuré sur le service

SQL : transactions par seconde



HTTP : requêtes terminées par seconde



20 clients, médiane de 3 essais	Avant	Après	Gain	Erreurs
Index composé : débit (tps)	14 840	25 133	×1,7	0
Index composé : p95 (ms)	2,8	1,5	−46 %	0
Chargement groupé : débit (req/s)	246,9	1032,1	×4,2	0
Chargement groupé : p95 (ms)	98,0	26,8	−73 %	0
Pagination, décalage 999 000 (ms)	944,3	0,177	×5 335	—
Fiche produit : miss → hit (ms)	2,7	0,988	×2,7	0

Débit du N+1 plafonné dès 5 clients (239 → 247 req/s), file du pool jusqu'à 15 requêtes ; groupé : 1032 req/s avec le même pool de 5. Pool 5 → 20 (N+1, 2 essais, indicatif) : attente 15 → 0, débit 137-216 → 276-299 req/s : corriger le N+1 vaut mieux qu'agrandir le pool. Charge ouverte (N+1) : stable à 50 % du débit fermé, instable à 90 %, effondrement (503) à 130 % ; la charge fermée (0 erreur) masque la saturation. Rapports concurrents : p95 +64 %, p99 ×2,2, médiane inchangée. work_mem 4 → 32 MB : agrégation mensuelle 84,0 → 53,5 ms (tri disque 3 040 kB → mémoire) ; effective_cache_size : aucun changement de plan. Un VACUUM est bloqué par une transaction longue (200 000 lignes mortes non supprimables, 200 000 supprimées après son COMMIT). SQL (pgbench) et HTTP (loadgen) ont des unités et des scénarios différents : leurs résultats restent séparés.

5. Tests de correction : invariants et fraîcheur

Invariant	Test	Résultat
Résultats identiques avant/après	empreinte md5 du résultat complet, ou différence symétrique entre deux requêtes (EXCEPT)	identiques pour toutes les lignes de la synthèse (agrégation 601f7d5aefe1, historique 21df3003a95c, file 7a826168f2fd, JSON 11157d484cf8, GiST e6f0c0d64f01)
Migration sans perte	EXCEPT ALL dans les deux sens sur id, email, nom ; NOT NULL ; rejet de NULL	0 écart ; erreur 23502 ; 0 NULL
Réponse API inchangée	corps JSON complet comparé, N+1 / groupé, OFFSET / curseur	identique à 5, 20, 50, 100 commandes ; 0 erreur sur toutes les séries
Export analytique	$\Sigma \text{nb_commandes} = \text{commandes payées}$; $\Sigma \text{montant} = \Sigma \text{commandes.total}$; deux exécutions	80 000 = 80 000 ; 23750126.25 = 23750126.25 ; md5 e17cb331709a = e17cb331709a
Fraîcheur du cache	UPDATE direct dans PostgreSQL, puis lecture par l'API (TTL 60 s)	l'API sert 57.50 (hit, 0 SQL) alors que la base a 19.90 ; PATCH par l'API : invalidation ok, valeur juste
Reprise et invalidation	PATCH pendant que Redis est arrêté ; expiration ; panne de Redis	invalidation = echouée : prix enregistré (34.90), copie ancienne conservée ; expiration TTL 60 → −2 ; repli sur PostgreSQL : 200, 1 SELECT par lecture
Course lecture / invalidation	reproduite à la main (psql + redis-cli)	l'API sert 34.90 alors que la base a 39.90 ; pistes : version, invalidation rejouée

Fraîcheur acceptée : catalogue, de quelques secondes à une minute (TTL 60 s) ; **aucun cache sur l'achat** : prix et stock contrôlés dans PostgreSQL, dans la transaction. Correction d'une journée d'export déjà publiée : une annulation tardive fait passer 2026-06-01 de 333 / 137287.50 à 332 / 136950.00 : la journée est republiée en entier. Fuseau : 4 896 commandes payées changeraient de jour entre UTC et Paris : la règle est UTC.

6. Décision finale : choix, limites et coût de maintenance

Retenu	Justification mesurée	Coût, risque ou condition de remise en cause
Index composé (client_id, created_at DESC, id DESC)	p95 2,8 → 1,5 ms, ×1,7 de débit à 20 clients ; client très actif ×91	WAL d'insertion +35 % ; à remettre en cause si l'import domine
Index partiel sur en_attente	×145, index 12,4 fois plus petit que l'index complet	ne sert que ce prédicat ; WAL +3,2 % (complet : +36,5 %)
Chargement groupé + curseur	21 → 2 requêtes ; pagination profonde constante (0,18 ms)	gain modeste en local ; curseur sans accès direct à la page n ; aucun gain à petite profondeur
Migration par étapes (NOT VALID, lots)	verrou exclusif 3 324 → 46,3 ms	plus lente au total ; lock_timeout et retour arrière à prévoir
Cache Redis sur la fiche produit seulement	2,7 → 0,988 ms, 0 SQL sur un hit	copie périmée jusqu'à 60 s ; invalidation à rendre fiable ; pas de cache sur l'achat
Rejeté	GIN sur 200 produits (jamais utilisé) ; index complet (12× plus gros) ; PgBouncer pour accélérer (40 → 5 connexions mais 261 → 94 tps) ; pool agrandi seul ; effective_cache_size (aucun effet)	variantes mesurées puis écartées

Conclusion bornée. Sur cette machine (8 cœurs, WSL2), ce jeu (100 000 commandes) et cette charge (1 à 20 clients, 10 s), l'index composé relève le débit SQL de ×1,7 et le chargement groupé le débit HTTP de ×4,2, sans erreur. Ce n'est pas « PostgreSQL devient plus rapide » : rien n'est établi sur la production, l'import, des clients très actifs ou plusieurs instances d'API. Coût de maintenance : un index de plus à maintenir (écriture, taille, VACUUM), une invalidation de cache à surveiller, des campagnes de mesure à rejouer (python3 atelier9/p2_charge.py, ≈ 16 min). Garanties de reprise : scripts DOWN, retrait des contraintes avant DROP COLUMN (qui détruit les valeurs), repli sur PostgreSQL si Redis tombe.

Résultat (slide 2)	Nature	Raison mesurée
Index composé, chargement groupé	optimisation reproductible	3 essais : 24 008-25 570 contre 14 570-15 555 tps ; groupé 978-1082 contre 238-253 req/s
Taille du pool, charge ouverte à 90 %	variation de mesure / instable	2 essais à pool 5 : 137 et 216 req/s ; un essai à 90 % tient, l'autre s'effondre : pas de médiane
Index (écriture), rapports concurrents, cache	déplacement du coût	WAL +35 % ; p95 des commandes +64 % ; copie périmée possible

Grille (slide 31)	Question de relecture	Réponse et preuve
Besoin	endpoint et charge définis ?	oui : GET /commandes?limit=20, 1 à 20 clients, 10 s ; autres accès listés en page 1
Correction	résultat et droits valides ?	oui : sections 3 et 5 (identité des résultats, 401 / 400, invariants d'export, 0 écart)
Méthode	test reproductible ?	oui : scripts atelier3 à atelier9, copie aux empreintes identiques, 3 essais alternés ; séries instables signalées
Intervention	le mécanisme explique-t-il le gain ?	oui : travail évité (21 → 2 requêtes, pages lues, relecture de table) ; plans et pages lues dans le PDF complet
Limites	coûts et risques exposés ?	oui : section 6 (écriture, fraîcheur, rapport concurrent, mesures locales, une machine)

Détail complet : Atelier1_Diagnostic_compact.pdf (ateliers 1 à 8) et test.pdf (+ Jour 5 avec annexes) ; scripts et sorties brutes : dossiers atelier3 à atelier9.